

Тестовое задание

Backend Trainee · Python / FastAPI

Уровень: Junior / Trainee Backend Developer

Стек: Python 3.11+, FastAPI, Pydantic v2, SQLAlchemy 2.0

Формат сдачи: GitHub-репозиторий + README

Это задание — не тест на знание алгоритмов. Мы смотрим на то, как ты думаешь: как структурируешь код, проектируешь API, обрабатываешь ошибки и пишешь сервисы, с которыми приятно работать другим людям.

Тебе предстоит собрать REST API для бронирования столика в ресторане — реальную фичу из нашего продукта, упрощённую до масштаба тестового задания. Сделай её так, как сделал бы в реальном проекте.

* Не старайся охватить всё. Лучше сдать меньше — но аккуратно и с тестами.

1. Задача

Реализуй REST API для онлайн-бронирования столика в ресторане. API должно позволять создавать бронь, получать список броней, получать бронь по id и отменять бронь.

Эндпоинты

Метод	Путь	Описание
POST	/bookings	Создать бронь. Валидирует данные, возвращает 201 и объект брони.
GET	/bookings	Список броней. Поддерживает фильтр по дате: ?date=2026-08-20.
GET	/bookings/{id}	Получить одну бронь по id. 404, если не найдена.
DELETE	/bookings/{id}	Отменить бронь (статус меняется на cancelled, запись не удаляется физически).

Данные брони

Поле	Тип	Правило
name	str	Имя гостя. Минимум 2 символа, только буквы, пробелы, дефис.
phone	str	Российский формат: +7XXXXXXXXXX или 8XXXXXXXXXX (10 цифр после кода).
booking_date	date	Не раньше сегодняшнего дня. Не позднее +90 дней.
booking_time	time	Только слоты: 12:00, 13:00 ... 22:00 (шаг 1 час).
guests	int	Целое число от 1 до 12.

2. Технические требования

Обязательный стек

- Python 3.11+
- FastAPI (роутинг, зависимости, обработка ошибок через HTTPException/exception_handler)
- Pydantic v2 — схемы запросов/ответов, валидация полей
- Хранение данных: SQLite через SQLAlchemy 2.0 (async) ИЛИ in-memory-хранилище — на твоё усмотрение, это не влияет на оценку
- Uvicorn как ASGI-сервер

Приветствуется, но не обязательно

- Alembic для миграций (если выбран SQLAlchemy)
- pytest + httpx.AsyncClient для тестов эндпоинтов
- Dockerfile / docker-compose для локального запуска
- Poetry или requirements.txt с зафиксированными версиями
- Разделение на слои: роуты / сервисы / схемы / модели

Структура проекта (ориентир)

```
app/  
  main.py  
  api/  
    bookings.py      # роуты  
  schemas/  
    booking.py       # Pydantic-схемы  
  services/  
    booking_service.py # бизнес-логика, валидация слотов  
  models/  
    booking.py       # ORM-модель или dataclass  
  core/  
    config.py  
  tests/  
    test_bookings.py
```

Пример схемы — schemas/booking.py

```
from datetime import date, time  
from pydantic import BaseModel, Field, field_validator  
  
class BookingCreate(BaseModel):  
    name: str = Field(min_length=2)  
    phone: str  
    booking_date: date  
    booking_time: time  
    guests: int = Field(ge=1, le=12)  
  
class BookingOut(BookingCreate):  
    id: int  
    status: str # 'active' | 'cancelled'
```

Пример валидатора телефона

```
import re

def validate_phone(value: str) -> str:
    digits = re.sub(r'\D', '', value)
    if len(digits) == 11 and digits[0] in ('7', '8'):
        return value
    raise ValueError(
        'Введите корректный номер: +7 или 8, 10 цифр'
    )
```

* Не нужно угадывать «правильный» способ хранения данных — важно, что API типизировано, валидирует вход и возвращает предсказуемые ошибки.

3. Валидация и обработка ошибок

Ошибки валидации должны возвращаться в стандартном формате FastAPI (422 Unprocessable Entity). Бизнес-ошибки — в формате {"detail": "..."}.

Ситуация	Код	Поведение
Успешное создание брони	201	Возвращает созданный объект брони с id и status=active.
Невалидные данные (телефон, дата, guests)	422	Стандартный ответ валидации FastAPI/Pydantic.
Бронь с указанным id не найдена	404	{"detail": "Booking not found"}
Слот на выбранные дату/время уже занят	409	Бонус: проверка пересечения броней по времени.
Успешная отмена брони	200	Возвращает бронь со status=cancelled.

Требование к документации

- Swagger UI должен быть доступен на /docs, ReDoc — на /redoc (стандартное поведение FastAPI, ничего дополнительно настраивать не нужно)
- Схемы Pydantic должны иметь понятные имена полей и, по желанию, примеры (Field(..., examples=[...]))

4. Сдача задания

Формат

- Создай публичный репозиторий на GitHub
- Приложи инструкцию запуска (локально и/или через Docker)
- Пришли ссылку на репозиторий рекрутеру

README.md — обязательно

- Как запустить проект локально (команды)
- Какие решения ты принял и почему (коротко, 3-5 предложений)
- Что бы ты доделал при наличии ещё времени

* README — часть оценки. Умение объяснить решения так же важно, как сам код.

Чего делать не нужно

- Подключать аутентификацию/авторизацию
- Разворачивать проект на реальном хостинге
- Использовать PostgreSQL или внешнюю БД (SQLite/in-memory достаточно)
- Добиваться 100% покрытия тестами (будет плюсом, но не обязательно)

5. Критерии оценки

Итоговая оценка — не сумма галочек. Мы смотрим на общее впечатление: насколько с кодом приятно работать и насколько ты думал о следующем разработчике, который откроет этот проект.

Критерий	Вес	На что смотрим
Качество кода	30%	Читаемость, декомпозиция на слои, отсутствие дублирования, осмысленные имена
Типизация и Pydantic-схемы	20%	Type hints везде, обоснованное использование Pydantic, нет необоснованных Any
Логика и валидация	20%	Все правила работают корректно, edge cases обработаны, слоты не пересекаются
Обработка ошибок и статус-коды	15%	Коды соответствуют ситуации, понятные сообщения об ошибках
README	15%	Понятное описание, объяснены решения, упомянуто что осталось бы доделать

Бonusные баллы

- Async SQLAlchemy 2.0 + Alembic-миграции
- pytest-тесты на сервисный слой и эндпоинты (httpx.AsyncClient)
- Dockerfile / docker-compose с рабочим запуском в одну команду
- Проверка пересечения слотов (409 Conflict) и пагинация списка броней